

MySQL慢SQL分析手册

1. 慢SQL发现与定位

1.1 开启慢查询日志

慢查询日志是分析性能问题的核心工具，建议在测试环境和生产环境均开启。

配置参数 (my.cnf)：

```
[mysqld]
# 开启慢查询日志
slow_query_log = ON
slow_query_log_file = /var/log/mysql/slow-query.log

# 慢查询阈值（秒），建议生产环境设置为1-2秒
long_query_time = 1

# 记录未使用索引的查询（强烈推荐开启）
log_queries_not_using_indexes = ON

# 记录慢查询的扫描行数阈值
min_examined_row_limit = 100

# 慢查询日志输出格式（FILE或TABLE，8.0推荐TABLE）
log_output = TABLE

# 每分钟记录多少条未使用索引的查询
log_throttle_queries_not_using_indexes = 10
```

动态修改（重启失效）：

```
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 1;
SET GLOBAL log_queries_not_using_indexes = ON;
```

1.2 查看慢查询统计

查看慢查询数量：

```
-- 查看慢查询总数（累计）
SHOW STATUS LIKE 'slow_queries';

-- 查看当前慢查询阈值设置
SHOW VARIABLES LIKE 'long_query_time';
```

查询mysql.slow_log表 (8.0)：

```
-- 查询最近10条慢SQL
SELECT start_time, query_time, lock_time, rows_examined, rows_sent, sql_text
FROM mysql.slow_log
ORDER BY start_time DESC
```

```

LIMIT 10;

-- 按平均执行时间排序
SELECT
    DIGEST_TEXT AS query_sample,
    COUNT_STAR AS exec_count,
    AVG_TIMER_WAIT / 1000000000 AS avg_time_ms,
    SUM_ROWS_EXAMINED AS total_rows_examined
FROM performance_schema.events_statements_summary_by_digest
ORDER BY avg_time_ms DESC
LIMIT 10;

```

1.3 使用pt-query-digest分析

pt-query-digest是Percona Toolkit中最常用的慢查询分析工具，能够对慢查询日志进行聚合统计。

基本用法：

```

# 分析慢查询日志文件
pt-query-digest /var/log/mysql/slow-query.log > slow_report.txt

# 分析最近24小时的慢查询
pt-query-digest --since '24h' /var/log/mysql/slow-query.log

# 分析并输出到数据库
pt-query-digest --review h=localhost,D=slow_log,t=query_review \
    /var/log/mysql/slow-query.log

```

输出解读：

- **Response time**：该SQL的总响应时间占比（最重要指标）
- **Calls**：执行次数
- **R/Call**：平均每次执行时间
- **95%**：95分位响应时间
- **Rows_examined**：扫描行数
- **Rows_sent**：返回行数
- **Rows_examined/Rows_sent**：扫描/返回比率，过高说明索引效率差

2. 执行计划分析（EXPLAIN）

2.1 EXPLAIN基础用法

```

-- 分析SELECT语句
EXPLAIN SELECT * FROM orders WHERE user_id = 12345;

-- 分析UPDATE/DELETE
EXPLAIN UPDATE orders SET status = 'done' WHERE user_id = 12345;

-- 显示更详细的信息（8.0）
EXPLAIN FORMAT=TREE SELECT * FROM orders WHERE user_id = 12345;

```

```
-- 分析实际执行的查询（包含优化器重写后的SQL）
EXPLAIN FORMAT=JSON SELECT * FROM orders WHERE user_id = 12345;
```

2.2 EXPLAIN输出字段详解

字段	含义	优化方向
id	SELECT执行顺序，越大越先执行	-
select_type	查询类型： SIMPLE/PRIMARY/SUBQUERY/DERIVED/UNION	避免过多的子查询和UNION
table	正在访问的表	-
type	访问类型（关键）	见下表
possible_keys	可能使用的索引	-
key	实际使用的索引	NULL表示未使用索引
key_len	使用的索引长度	越长说明使用了越多的索引列
ref	索引列与哪个值/列进行比较	const表示常量
rows	预估需要扫描的行数	越小越好
filtered	过滤后剩余行的百分比	越高越好
Extra	额外信息（关键）	见下表

2.3 type类型（从好到差）

type	含义	是否可用	优化建议
system	系统表，只有一行	理想	-
const	主键或唯一索引等值查询	理想	-
eq_ref	连接查询时，被驱动表使用主键/唯一索引	优秀	-
ref	使用非唯一索引的等值查询	良好	-
range	索引范围扫描（BETWEEN/IN/>等）	可接受	尽量缩小范围
index	全索引扫描	较差	减少SELECT *，使用覆盖索引
ALL	全表扫描	糟糕	必须添加索引

2.4 Extra信息解读

Extra信息	含义	优化建议
Using where	使用WHERE条件过滤	正常
Using index	使用了覆盖索引（不回表）	理想状态
Using index condition	使用了索引下推（5.6+）	良好
Using where; Using index	通过索引直接获取数据	优秀
Using temporary	使用了临时表（常见于GROUP BY/ORDER BY）	需要优化，添加索引
Using filesort	需要额外排序操作	需要优化，排序字段建索引
Using join buffer	连接查询未使用索引	为连接字段添加索引
Impossible WHERE	WHERE条件永远为假	检查业务逻辑
No matching row	未找到匹配行	检查数据或业务逻辑
Select tables optimized away	聚合函数直接返回（如MIN/MAX使用索引）	理想

2.5 实战案例：分析一条慢查询

问题SQL：

```
SELECT u.name, o.order_no, o.amount
FROM users u
JOIN orders o ON u.id = o.user_id
WHERE u.created_at > '2024-01-01'
      AND o.status = 'paid'
ORDER BY o.created_at DESC
LIMIT 20;
```

EXPLAIN结果：

id	select_type	table	type	possible_keys	key	rows	Extra
1	SIMPLE	u	ALL	PRIMARY	NULL	500000	Using where; Using filesort
1	SIMPLE	o	ALL	NULL	NULL	1000000	Using where; Using join buffer

问题诊断：

- 1. **users表全表扫描** (type=ALL) ， 扫描50万行
- 2. **orders表全表扫描** (type=ALL) ， 扫描100万行
- 3. **Using filesort**： 需要额外排序
- 4. **Using join buffer**： 连接查询未使用索引

优化方案：

```
-- 1. 为users.created_at添加索引
ALTER TABLE users ADD INDEX idx_created_at (created_at);

-- 2. 为orders.user_id和orders.status、orders.created_at添加复合索引
ALTER TABLE orders ADD INDEX idx_user_status_created (user_id, status,
created_at);

-- 3. 改写SQL，先过滤orders表
SELECT u.name, o.order_no, o.amount
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.status = 'paid'
      AND u.created_at > '2024-01-01'
ORDER BY o.created_at DESC
LIMIT 20;
```

3. 索引优化策略

3.1 索引类型选择

索引类型	适用场景	注意事项
B-Tree (默认)	等值查询、范围查询、排序、分组	绝大多数场景首选
Hash (MEMORY引擎)	精确等值查询	不支持范围查询、排序
全文索引	全文搜索 (LIKE '%keyword%')	有专门语法MATCH AGAINST
空间索引	GIS地理数据	使用SPATIAL关键字

3.2 复合索引设计原则（最左前缀）

核心原则： MySQL从左到右使用索引列，跳过某一系列后，后面的列无法使用索引。

```
-- 创建复合索引
CREATE INDEX idx_user_status_time ON orders (user_id, status, created_at);

-- 能使用索引的查询:
WHERE user_id = 1 -- ✅ 使用第一列
WHERE user_id = 1 AND status = 'paid' -- ✅ 使用前两列
WHERE user_id = 1 AND status = 'paid' AND created_at > '2024-01-01' -- ✅ 全使用
WHERE user_id = 1 AND created_at > '2024-01-01' -- ⚠️ 只使用user_id, 跳过status

-- 不能使用索引的查询:
WHERE status = 'paid' -- ❌ 没有从最左列开始
WHERE created_at > '2024-01-01' -- ❌ 没有从最左列开始
```

复合索引排序规则：

```
-- 索引定义: INDEX idx_col1_col2 (col1, col2)

-- ✅ 可以利用索引排序
ORDER BY col1, col2
ORDER BY col1 DESC, col2 DESC

-- ❌ 无法利用索引排序
ORDER BY col2, col1 -- 顺序不一致
ORDER BY col1 ASC, col2 DESC -- 方向不一致
ORDER BY col1 DESC, col2 ASC
```

3.3 索引失效场景（必查清单）

场景	错误示例	正确示例
对索引列使用函数	WHERE DATE(created_at) = '2024-01-01'	WHERE created_at >= '2024-01-01' AND created_at < '2024-01-02'
隐式类型转换	WHERE user_id = '123' (user_id是INT)	WHERE user_id = 123
LIKE以%开头	WHERE name LIKE '%张三%'	WHERE name LIKE '张三%' (前缀匹配)
OR条件	WHERE user_id = 1 OR status = 'paid'	拆分成UNION ALL
使用!=或<>	WHERE status != 'deleted'	考虑改写或接受全表扫描
使用NOT IN	WHERE id NOT IN (1,2,3)	使用NOT EXISTS或LEFT JOIN
索引列参与计算	WHERE age + 1 = 30	WHERE age = 29

3.4 覆盖索引 (Using Index)

定义： 查询需要的所有列都在索引中，无需回表查询数据行。

示例：

```
-- 假设有索引 idx_user_status (user_id, status)
-- 这个查询只需要user_id和status，索引中已有，无需回表
SELECT user_id, status FROM orders WHERE user_id = 12345; -- ✅ Using index

-- 这个查询还需要amount字段，索引中没有，需要回表
SELECT user_id, status, amount FROM orders WHERE user_id = 12345; -- ⚠️ 回表
```

优化： 将查询所需字段放入索引（注意索引长度不要过大）：

```
-- 创建覆盖索引
CREATE INDEX idx_user_status_amount ON orders (user_id, status, amount);
```

3.5 索引创建与删除

```
-- 查看表已有索引
SHOW INDEX FROM orders;

-- 创建普通索引
CREATE INDEX idx_user_id ON orders (user_id);

-- 创建唯一索引
CREATE UNIQUE INDEX idx_order_no ON orders (order_no);

-- 创建复合索引
CREATE INDEX idx_user_status_time ON orders (user_id, status, created_at);

-- 删除索引
DROP INDEX idx_user_id ON orders;

-- 修改表添加索引（生产环境建议使用pt-online-schema-change）
ALTER TABLE orders ADD INDEX idx_user_id (user_id);
```

4. SQL改写优化技巧

4.1 分页优化 (LIMIT深翻页)

问题： `LIMIT 100000, 20` 需要扫描100020行，越往后越慢。

优化方案1：延迟关联

```
-- 原SQL
SELECT * FROM orders ORDER BY id LIMIT 100000, 20;

-- 优化后（先查主键，再关联）
SELECT * FROM orders o
INNER JOIN (
    SELECT id FROM orders ORDER BY id LIMIT 100000, 20
) t ON o.id = t.id;
```

优化方案2：游标分页（记住上一页的最后ID）

```
-- 上一页最后一条记录的id是100000
SELECT * FROM orders WHERE id > 100000 ORDER BY id LIMIT 20;
```

4.2 COUNT(*)优化

```
-- ❌ 慢: COUNT(DISTINCT)
SELECT COUNT(DISTINCT user_id) FROM orders;

-- ✅ 快: 先GROUP BY再COUNT
SELECT COUNT(*) FROM (SELECT user_id FROM orders GROUP BY user_id) t;

-- ❌ 慢: COUNT(*)带复杂条件
SELECT COUNT(*) FROM orders WHERE status = 'paid' AND created_at > '2024-01-01';

-- ✅ 快: 使用近似值或汇总表
-- 方案1: 使用EXPLAIN的rows列（近似值）
EXPLAIN SELECT * FROM orders WHERE status = 'paid';
-- 方案2: 维护汇总表
CREATE TABLE order_stats (status VARCHAR(20), cnt INT, update_time DATETIME);
```

4.3 子查询优化（改JOIN）

```
-- ❌ 慢: 相关子查询（外层每行执行一次子查询）
SELECT * FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id AND o.amount > 1000);

-- ✅ 快: 改为JOIN
SELECT DISTINCT u.* FROM users u
INNER JOIN orders o ON u.id = o.user_id
WHERE o.amount > 1000;
```

```
-- ❌ 慢: IN子查询（MySQL 5.6前优化器不友好）
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders WHERE amount > 1000);

-- ✅ 快: 改为JOIN
SELECT u.* FROM users u
INNER JOIN orders o ON u.id = o.user_id
WHERE o.amount > 1000;
```


4.4 UNION vs UNION ALL

```
-- UNION: 去重, 有额外排序开销
SELECT user_id FROM orders WHERE status = 'paid'
UNION
SELECT user_id FROM orders WHERE status = 'shipped';

-- UNION ALL: 不去重, 无排序开销 (更推荐)
SELECT user_id FROM orders WHERE status = 'paid'
UNION ALL
SELECT user_id FROM orders WHERE status = 'shipped';
```

4.5 批量操作优化

```
-- ❌ 慢: 循环单条插入 (应用层循环)
INSERT INTO orders (order_no, amount) VALUES ('NO001', 100);

-- ✅ 快: 批量插入
INSERT INTO orders (order_no, amount) VALUES
('NO001', 100),
('NO002', 200),
('NO003', 300);

-- ❌ 慢: 循环单条更新
UPDATE orders SET status = 'paid' WHERE id = 1;
UPDATE orders SET status = 'paid' WHERE id = 2;

-- ✅ 快: 使用CASE WHEN批量更新
UPDATE orders SET status = CASE id
    WHEN 1 THEN 'paid'
    WHEN 2 THEN 'paid'
END
WHERE id IN (1, 2);
```

5. 慢SQL监控与告警

5.1 实时监控慢SQL

```
-- 查看当前正在执行的慢查询 (执行时间超过10秒)
SELECT
    id,
    user,
    host,
    db,
    command,
    time AS exec_time_sec,
    state,
    info AS sql_text
FROM information_schema.PROCESSLIST
WHERE command != 'sleep'
    AND time > 10
```

```
ORDER BY time DESC;
```

5.2 设置告警阈值

使用pt-query-digest持续监控：

```
# 每5分钟分析一次慢查询日志，超过阈值则告警
pt-query-digest /var/log/mysql/slow-query.log \
  --since '5m' \
  --filter '$event->{arg} =~ m/^select/i' \
  --report \
  --limit 10
```

5.3 慢查询自动化处理脚本

```
#!/bin/bash
# slow_query_killer.sh - 自动杀掉执行时间超过阈值的查询

THRESHOLD=60 # 60秒
LOG_FILE="/var/log/slow_query_killer.log"

mysql -e "SELECT CONCAT('KILL ', id, ';') FROM information_schema.PROCESSLIST
WHERE command != 'sleep' AND time > $THRESHOLD" \
| grep -v "CONCAT" \
| while read kill_cmd; do
    echo "$(date): Executing $kill_cmd" >> $LOG_FILE
    mysql -e "$kill_cmd"
done
```

6. 优化案例库

6.1 案例1：分页查询从3秒优化到50ms

问题SQL：

```
SELECT * FROM operation_log
WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31'
ORDER BY id DESC
LIMIT 10000, 20;
```

分析：扫描所有符合时间范围的行，然后排序，再取第10000-10020行。

优化方案：

```
-- 1. 创建复合索引
ALTER TABLE operation_log ADD INDEX idx_created_id (created_at, id);

-- 2. 改写SQL（延迟关联）
SELECT * FROM operation_log o
INNER JOIN (
    SELECT id FROM operation_log
    WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31'
    ORDER BY id DESC
    LIMIT 10000, 20
) t ON o.id = t.id;
```

6.2 案例2：聚合报表从10秒优化到0.5秒

问题SQL：

```
SELECT
    DATE(created_at) AS dt,
    COUNT(*) AS order_cnt,
    SUM(amount) AS total_amount
FROM orders
WHERE created_at BETWEEN '2024-01-01' AND '2024-01-31'
GROUP BY DATE(created_at);
```

分析：对created_at使用DATE函数，导致无法使用索引。

优化方案：

```
-- 1. 修改查询条件，避免函数
SELECT
    DATE(created_at) AS dt,
    COUNT(*) AS order_cnt,
    SUM(amount) AS total_amount
FROM orders
WHERE created_at >= '2024-01-01' AND created_at < '2024-02-01'
GROUP BY DATE(created_at);

-- 2. 如果还是慢，创建汇总表（空间换时间）
CREATE TABLE order_daily_stats (
    stat_date DATE PRIMARY KEY,
    order_cnt INT,
    total_amount DECIMAL(10,2),
    update_time DATETIME
);

-- 使用事件每天凌晨汇总
INSERT INTO order_daily_stats (stat_date, order_cnt, total_amount, update_time)
SELECT
    DATE(created_at),
    COUNT(*),
    SUM(amount),
    NOW()
FROM orders
WHERE DATE(created_at) = CURDATE() - INTERVAL 1 DAY
```

```
GROUP BY DATE(created_at)
ON DUPLICATE KEY UPDATE
    order_cnt = VALUES(order_cnt),
    total_amount = VALUES(total_amount),
    update_time = VALUES(update_time);
```

7. 快速诊断清单

当遇到慢SQL问题时，按以下顺序排查：

步骤	检查项	命令/方法
1	是否有全表扫描？	<code>EXPLAIN</code> 查看type是否为ALL
2	是否使用索引？	<code>EXPLAIN</code> 查看key是否为NULL
3	是否有Using filesort？	<code>EXPLAIN</code> 查看Extra
4	是否有Using temporary？	<code>EXPLAIN</code> 查看Extra
5	预估扫描行数是否过大？	<code>EXPLAIN</code> 查看rows
6	是否在索引列上使用了函数？	检查WHERE条件
7	是否隐式类型转换？	比较字段类型和传入值类型
8	是否深分页？	检查LIMIT offset值
9	表统计信息是否过期？	<code>ANALYZE TABLE</code>
10	是否有锁等待？	<code>SHOW PROCESSLIST</code> 查看State